

DOTS & BOXES

Contents

1. Abstract:	2
2. Introduction:	2
3. Project Overview:	2
4. . Objectives:	2
5. Tools and Technologies:	3
6. System Requirements:	3
7. Game Rules:	3
8. System Design and Data Structures:	4
9. Main Methods and Their Functions:	4
10. Game Working Process:	4
11. User Interface Description:	5
12. Testing:	5
13. Output:	6
14. Limitations:	6
15. Future Improvements:	6
16. Conclusion:	7

1. Abstract

The Dots & Boxes project is a two-player desktop game developed using Java and Java Swing. The game board contains nine boxes arranged in three rows and three columns. Players take turns drawing horizontal or vertical lines between neighboring dots. When a player completes the fourth side of a box, that box is assigned to the player and one point is added to their score.

The application includes a graphical board, score display, current-turn label, player-specific box colors, a Restart Game button, and a game-over message. The project demonstrates the use of two-dimensional arrays, conditional statements, loops, methods, mouse events, and graphical user interface components.

2. Introduction

Dots & Boxes is a strategy game in which players connect dots to form boxes. This version is implemented as a graphical application, so users interact with the board by clicking with the mouse instead of entering commands in a console.

The Java Swing interface displays the game board and important information. The program records drawn lines, identifies completed boxes, updates scores, switches turns when required, and determines when the game is finished.

3. Project Overview

Project Name: Dots & Boxes

Course Name: Software Development

Programming Language: Java

GUI Technology: Java Swing

Main Class: DOTSANDBOX

Game Type: Two-player, turn-based desktop game

Board Size: 3×3 boxes

4. Objectives

Develop a playable Dots & Boxes game in Java.

Create a graphical interface using Java Swing.

Draw dots, horizontal lines, vertical lines, and completed boxes.

Allow two players to make moves using mouse clicks.

Detect when a box is completed and assign it to the correct player.

Count and display each player's score.

Manage turns according to the game rules.

Detect the end of the game and display the result.

Allow users to restart the game without closing the application.

5. Tools and Technologies

Java: Used to implement the game rules and program logic.

Java Swing: Used to create the application window, panels, labels, and button.

Java AWT Graphics and Graphics2D: Used to draw the board, dots, lines, colors, and game-over overlay.

Mouse events: Used to detect where a player clicks on the board.

Two-dimensional arrays: Used to store drawn horizontal lines, vertical lines, and box ownership.

JDK and a Java IDE/editor: Required to compile and run the application.

6. System Requirements

Hardware: A desktop or laptop computer with a screen, keyboard, and mouse.

Software: Java Development Kit (JDK), a compatible operating system, and a Java IDE or code editor.

Runtime: The computer must support Java Swing and AWT.

7. Game Rules

1. The board contains nine boxes arranged in a 3×3 grid.
2. Players alternate drawing one horizontal or vertical line between neighboring dots.
3. A line that has already been drawn cannot be selected again.
4. A box is completed when all four of its sides have been drawn.
5. The player who completes a box receives one point and ownership of that box.
6. If a player completes a box, they keep their turn.
7. If a move does not complete a box, the turn switches to the other player.
8. The game ends when all nine boxes are owned.
9. The player with the higher score wins. If the scores are equal, the result is a draw.

8. System Design and Data Structures

The main class, DOTSANDBOX, extends JFrame and creates the application window. It also creates the top panel containing the score label, turn label, and Restart Game button. The inner GamePanel class draws the board and listens for mouse clicks.

The main data structures are:

$h[n + 1][n]$ — a boolean two-dimensional array that records horizontal lines.

$v[n][n + 1]$ — a boolean two-dimensional array that records vertical lines.

$boxOwner[n][n]$ — an integer two-dimensional array that records who owns each box: 0 means unclaimed, 1 means Player 1, and 2 means Player 2.

$p1$ and $p2$ — integer variables that store the players' scores.

$player$ — identifies the current player, either 1 or 2.

$gameOver$ — a boolean variable that prevents additional moves after the game ends.

$cellSize$, $startX$, and $startY$ — control the board's dimensions and position.

9. Main Methods and Their Functions

$DOTSANDBOX()$ — Initializes the window, sets its title and size, creates the top panel and game panel, and connects the restart button to the restart method.

$paintComponent(Graphics g)$ — Draws the gradient background, completed boxes, dots, horizontal and vertical lines, and the game-over message.

$handleClick(int x, int y)$ — Converts the mouse position into a board location, checks whether a valid undrawn edge was selected, records the move, and runs the game logic.

$checkBoxes()$ — Checks each unclaimed box to see whether its four sides are present. It assigns completed boxes to the current player and returns the number of boxes completed by the move.

$gameFinished()$ — Checks all boxes. It returns true when every box has an owner and false if at least one box remains incomplete.

$updateUI()$ — Updates the score label and current-turn label and changes the turn-label color according to the current player.

$restartGame()$ — Clears all stored lines and box owners, resets both scores and the current player, clears the game-over state, and refreshes the interface.

$main(String[] args)$ — Starts the Swing application using `SwingUtilities.invokeLater()`.

10. Game Working Process

Step 1: The program starts and displays the Dots & Boxes window.

- Step 2: Player 1 begins the game, and the current turn is displayed at the top.
- Step 3: The player clicks near an available horizontal or vertical edge to draw a line.
- Step 4: The program records the line and checks whether any box has been completed.
- Step 5: If a box is completed, it is colored for the current player and the player's score increases. That player gets another turn.
- Step 6: If no box is completed, the turn switches to the other player.
- Step 7: The program checks whether all boxes have been claimed and updates the score and turn display.
- Step 8: When all boxes are completed, the game-over overlay displays the winner or a draw.
- Step 9: Clicking Restart Game resets the board and allows a new game to begin.

11. User Interface Description

The top section displays Player 1's score, Player 2's score, the current turn, and the Restart Game button. The game board has a soft gradient background with black dots and black lines.

Completed boxes are filled with pink-red for Player 1 and blue for Player 2. The interface uses anti-aliasing to make graphics appear smoother. When the game ends, a light overlay displays “GAME OVER” and the final result.

12. Testing

Test 1 — Start-up: Run the program and check that the window, board, score, turn label, and restart button appear.

Test 2 — Valid line: Click near an available edge and confirm that the line is drawn.

Test 3 — Repeated line: Click an edge that is already drawn and confirm that it is not added a second time.

Test 4 — Box completion: Draw the fourth side of a box and confirm that the box receives the current player's color.

Test 5 — Score update: Confirm that each completed box adds one point to the correct player's score.

Test 6 — Turn handling: Confirm that a player keeps the turn after completing a box and otherwise the turn changes.

Test 7 — Game completion: Complete all boxes and check that the game-over message and result appear.

Test 8 — Restart: Click Restart Game and confirm that all lines, box colors, scores, and game-over status reset.

13.output:



14. Limitations

The board is fixed at 3×3 boxes.

The game supports two local players sharing one computer.

There is no computer-controlled opponent or online multiplayer.

The application does not save scores or match history permanently.

Mouse clicks must be close enough to an available edge to be recognized.

15. Future Improvements

Allow users to select different board sizes.

Add a computer-controlled opponent.

Add sound effects and optional background music.

Save match results and player statistics.

Improve click detection so selecting an edge is easier.

Add a start screen and additional visual themes.

16. Conclusion

The Dots & Boxes project shows how Java and Swing can be used to develop an interactive desktop game. It combines arrays, loops, conditional logic, methods, mouse-event handling, and custom graphics. The score system, turn management, completed-box detection, game-over result, and restart feature work together to implement the main rules of the game. The project provides practical experience in building, testing, and documenting a graphical Java application.